摘要：

SpaceX技术骨干Lee Robinson在斯坦福发表演讲指出，AI正步入“全天候待命”的Agent时代。在盲测中AI审核命令的准确率已超越人类，开发者正从“审批者”变为“指导者”，AI代理将在未来6至12个月内实现连续数月自主工作。更残酷的市场法则是：未来所有新产品若不能成为Agent的交互入口，终将沦为其他AI系统底层的数据养料，而支撑Agent运行的底层基建将催生出万亿级的新商业赛道。

在斯坦福大学的课堂上，SpaceX的Lee Robinson揭示了AI代理正从被动响应向“全天候待命”进化，而在这个万亿级的新赛道中，任何未能掌握Agent入口的产品，终将彻底沦为其他AI系统的数据“养料”。

近日，在斯坦福大学CS146S课程上，SpaceX技术骨干（前Cursor核心成员）Lee Robinson发表了关于“全天候代理（Always-on Agent）工作原理”的主题演讲。

在当前市场高度关注AI商业化落地、算力成本控制以及应用层爆发节点的背景下，Robinson的演讲不仅拆解了前沿AI架构的底层逻辑，更向业界抛出了一个极具冲击力的商业论断：未来的SaaS和软件生态将被彻底重构。开发者的角色正从“审批每一条命令”转变为“指导代理持续运行”。



AI软件的终局：“不做入口，就沦为无头数据”

在探讨AI应用的未来想象空间时，市场一直在寻找下一代超级入口。Robinson引用了OpenRouter成员的洞察指出，现在所有的应用看起来都一样：侧边栏、代理、聊天框。“这就像2005年一样，大家都在做同样的事情，有服务器、有数据库、有CRUD界面。”

但他强调，这已经成为一种新常态。对于现有的科技公司和SaaS企业而言，残酷的商业逻辑已经浮现：**“每个新产品都必须具备某种代理功能，要么集成到现有系统中，要么就沦为供其他代理使用的无头数据（headless data）。”**

这意味着，在未来的商业版图中，应用层的价值将极大取决于其是否能成为Agent的交互中枢。围绕这一趋势，从持久工作流、云端计算机、内存管理到技能评估等每一个细分领域，“都相当于数十亿甚至数万亿美元的风险投资机会，可以说是下一代公司、下一代产品的基础组件。”

生产力范式跃迁：盲测中AI审核已超越人类

市场极为关注AI Agent能否真正取代人力并形成可靠的业务流。Robinson将开发者与AI的合作划分为四个时代：从最初的复制粘贴ChatGPT代码，到简单的终端交互，再到专用应用程序，直到如今迈入拥有独立计算机和内存的“始终在线”形态。

在这个新时代，Agent更像是一个“全天候待命的同事”。Robinson指出，“它不再只是在问答式的环境中回答问题，而是可以执行更复杂的任务……它知道什么时候该保持沉默。”

更重要的是，阻碍Agent大规模部署的“安全性与准确性”痛点正在被攻克。在从“人工审批”向“自动审核”过渡的过程中，AI系统本身的审核能力已经迎来了质变。“如果你做一个盲测，对比AI模型和人工审核命令的结果，大多数情况下，模型实际上做得更好。” 这种转变使得代理可以在后台自动运行数小时甚至数天，极大地拓宽了AI赋能企业级任务的想象空间。

重塑成本与架构：打造“无限上下文”的错觉

对于投资者和企业而言，大模型的Token消耗直接挂钩着利润率。如何以最低的成本实现最强的代理功能？Robinson给出的答案是“瘦客户端固定服务器架构”以及“主子代理协同”。

在Grok Bot等前沿框架中，“只有一个工具用于在客户端和服务器之间进行通信”。在执行任务时，代理会遵循一套极其严苛的成本阶梯：它会先尝试基于历史对话查找，再尝试网络搜索或后台操作，“先尝试最便宜或最可靠的选项”，只有在所有方法失败后才会去打扰用户。

为了突破模型上下文窗口的物理限制，主代理会将繁重的工作交给拥有全新工作记忆的子代理（辅助程序）。“主代理将繁重的工作交给子代理，这就是如何打造出感觉拥有无限上下文的产品。”

同时，对于基础设施层，Robinson提出了一种极端的成本控制理念：在动辄消耗数十万Token的长期对话中，保持提示信息的缓存至关重要。“如果出现缓存未命中，实际上就相当于系统出现了一个bug。” 趁缓存还热的时候立即压缩长对话，是降低后续成本的关键。

“删除产品”法则：未来6到12个月代理将能连续工作数月

站在当前的时点，市场迫切想知道AI应用的下一步爆发点何时到来。

Robinson分享了一个在Cursor和SpaceX都秉持的核心开发原则——“删除产品”。这意味着开发者必须接受一个现实：“假设今天的模型是它们有史以来最糟糕的。” 面对必然会快速迭代的底层大模型，最聪明的策略是尽可能少地构建繁杂的UI，而是去打造一个极其简单的界面，让模型替你思考和行动。

对于未来的业绩指引和技术时间表，Robinson给出了极为乐观的预期。他认为在未来六到十二个月内，随着模型长时间执行指令能力的提升和持久工作流（如Temporal开源工具）的完善，“代理工作数周或数月的情况很快就会出现”。

这不仅意味着企业端效率的指数级提升，也对整个AI行业提出了新的挑战：当AI代理每月更新一次模型，但代理本身需要连续运行一个月时，传统的评估体系将面临重构。但这正是下一轮技术红利与巨大商业价值的藏身之处。

> **以下是演讲文稿**
>
> 我是 Lee。今天我们将简单聊聊这种新型的始终在线、主动式、持久型代理的工作原理，并帮助大家了解它们的构建基础。顺便提一下，我们今天要谈到的一些内容目前还在研发中，是

SpaceX 正在努力的方向。

在第一周，大家搭建了一个非常简单的框架，帮助你们理解核心概念：模型可以循环调用工具，这非常强大。它为模型解锁了许多新功能，但也存在一些缺点，例如，当你合上笔记本电脑时，进程就会停止运行。我想谈谈如何将这个简单但强大的模型优化思路扩展到这种新型代理——始终在线的代理。

它不只是在你的终端运行，而是可以在服务器上运行，可以在云端运行。你可以关闭电脑，它也可以重启，即使电脑崩溃，它也能正常运行。**这个代理程序拥有自己的独立计算机，它不是你的笔记本电脑，而是一台云端的 Linux 服务器，上面有它自己的桌面、文件、浏览器等等，可能还有一些 3D 应用。**它可以将文件保存到这台服务器上。随着时间的推移，它可以更好地了解你的操作过程，并建立起你使用方式的记忆。

与终端中那些会响应你发送的每条消息的程序不同，这种始终在线的代理更像是你的同事，它不会总是立即发送消息，它知道何时应该保持沉默。有时，它可能只想在后台运行一段时间，只有在有重要更新时才会回复你。而且，它的触发方式也不仅仅局限于你的输入，并非只有你在终端中才能触发它。也许是 Slack 消息、电子邮件或其他类型的事件唤醒了这个代理去执行任务。

理想情况下，我们希望这类始终在线的代理能够长时间工作，不仅仅是几个小时或几天，而是几周甚至几个月。这需要一种略有不同的架构，我们稍后会详细介绍。我们将深入探讨我们如何构建这种架构，如何搭建框架，以及我们今天所取得的成就和未来的发展方向。我们还会详细介绍我们如何设计这个系统，使其能够主动、持久地执行任务并很好地遵循指令。当然，最后会留出提问时间，如果您有任何关于今天内容之外的问题，都可以提问。

人工智能在盲测中基本能够击败人类

"首先，我们是如何走到今天的。我认为，到目前为止，开发者与人工智能的合作大致经历了四个阶段。第一个阶段是从 ChatGPT 复制粘贴代码，然后放到你选择的编辑器里，现在感觉好像是几十年前的事了，但实际上只是几年前的事。"

然后，当模型能够更好地实现我们在上一讲中提到的那种简化框架时，它们就能编辑文件、运行 shell 命令，并通过终端中非常简单的界面与之交互。这迅速发展起来，因为模型突然拥有了这种新功能，可以为你做更多主动工作，从而提高效率。

第三阶段是将编码代理（它们上手比较简单）开发成专用应用程序。此时，人们开始并行运行多个代理，他们可能在云端运行这些代理，而不仅仅是在本地计算机上。但即便如此，你仍然会成为瓶颈，因为你需要审查它们运行的每一个命令，需要确保每个 shell 命令都正确无误，需要密切关注它们，了解所有情况。

如今，随着模型、工具和产品的改进，我们进入了一个新时代，智能体开始以"始终在线"的形式运行。它们可以在后台自动运行，拥有独立的计算机和内存，能够边工作边学习新技能，即使你睡觉或去做其他事情，它们也能继续工作。一个很好的例子是，我们已经不再需要每天手动提示智能体，尽管有时仍然需要这样做，但你可以设置智能体，让它在后台自动运行。

所以，以前可能是这样的：你给代理发出指令，然后等待，再给代理发出指令，再等待。如果它要求你运行 shell 命令，你必须批准或禁止该命令。而现在，你不再需要排队等待命令，而是可以引导代理。比如你可以说："嘿，去看看这个线程。"然后紧接着又说："哦，对了，这件事应该在周四做。"以前的模型在你立即打断并发出另一个命令时会感到困惑，但现在它们能够理解，就像你和朋友聊天一样，它们可以分辨出这两件事之间的关联。模型和框架现在都能处理这种情况了。

如果你说"嘿，去发送这封邮件，去发送这条 Slack 消息"，系统会启动额外的安全检查，很多公司称之为"自动审核"，它本质上是一个独立的 AI 模型，会检查所有运行的命令。这实际上比人

工审核数百条 shell 命令并逐一批准要安全得多。我们现在已经达到了这样的程度：如果你做一个盲测，对比 AI 模型和人工审核命令的结果，大多数情况下，模型实际上做得更好。

始终在线的代理就像给一个真正在工作的朋友发短信

"有点矛盾的是，随着模型的改进，界面反而变得更加简洁了。这种始终在线的代理，我们称之为机器人。现在涌现出很多这类新产品，并且发展迅猛。我认为主要原因在于我们都会发短信，每个人都非常擅长使用手机。我们已经将这种与人工智能交互的方式简化到就像在和朋友发短信一样，但它实际上可以为你完成真正的工作。它不再只是在问答式的环境中回答问题，而是可以执行更复杂的任务，比如你通常会交给同事完成的任务。"

这些改进主要体现在模型本身。我将重点介绍其中六点，但实际上还有更多。

首先，在过去一两年里，这些模型在长时间执行指令方面有了显著提升，它们不会出现混乱或错误。现在，只要压缩和精简得当，模型就能稳定运行数小时甚至数天，我们稍后会详细讨论这一点。

第二，模型现在可以有效地将任务交给辅助程序，这些辅助程序被称为子代理。子代理之所以有用，是因为模型拥有一个记录当前进程的工作记忆，而这个记忆最终会被填满。因此，如果模型能够将任务交给拥有全新工作记忆的辅助程序，那就非常有利了。这实际上可以为模型提供一个无限的上下文，使其能够执行一些在令牌使用或工具调用方面可能比较耗时的任务，然后再返回给主代理并报告更新。稍后我会提供一些相关的图示。

现在的模型调用工具的能力更强了。就在几年前，它们基本上还无法很好地调用工具，而且经常出现幻觉。有趣的是，现在人们很少再谈论幻觉了，因为这基本上已经是一个解决了的问题。不仅如此，你还可以给模型提供非常强大的工具，就像给新成员入职一样。这当然包括运行 shell 命令和文件，但你还可以给它们一台完整的 Linux 计算机，允许它们制作演示视频等等。它们可以像人一样在屏幕上点击操作，而且它们在这方面（也就是所谓的计算机使用）越来越熟练。这基本上意味着，任何没有公开 API 或 MCP 服务器的 SaaS 产品或网站，都可以通过这些模型和机器人的计算机使用能力，实现无人值守的自主操作。

你可以把所有信息都存储在文件中，这样做的好处是模型可以非常轻松地调用和搜索这些信息，我们稍后会详细讨论。最后，你还可以教机器人如何完成一些工作，它会将这些工作转化为技能、记忆、例程或其他可以重复使用的东西。

代理所掌握的知识以纯文本文件的形式存在，它可以搜索这些文件；而技能则是将某人针对特定任务的专业知识编码到 Markdown 文件中。

"关于文件管理，这只是个简化的例子，但你可以想象，当你使用机器人时，它的底层架构大概是这样的：**你会建立起对用户的画像或理解，这些信息会体现在每一条提示信息中。比如，你可能喜欢小写字母的文本格式，那么每条消息都应该使用小写字母。此外，随着你使用这些机器人的时间推移，你会建立起类似工作日志的东西，就像你和同事一起工作，他们每天记录工作内容一样。所有这些信息都会被保存到文件中，供模型以后搜索或使用。**比如，如果你问'嘿，我上周做了什么'，它就可以直接查找。这些模型非常擅长使用 Linux 命令，运行 shell 命令来进行 grep 搜索，并浏览这些文件。"

同样，技能本质上就是将你完成特定任务的知识编码成一个 Markdown 文件，其中包含了你需要执行的具体步骤，以及你认为决定成败的关键因素。这不仅仅指你自己的技能。现在，已经形成了一个完整的生态系统，许多公司和开发者都在编写这些技能，例如如何发送一封优秀的电子邮件、如何构建一个出色的 API 等等。

举个例子，Grok Bot 就有一个完整的插件系统。基本上，任何你想使用的工具，都有相应的插件可以安装，这些插件会用到技能或服务器。此外还有例程，如果你熟悉 cron 作业，就会知道

例程是按照某种计划运行一些提示符，这也非常有用。基本上，一切都与计算机有关。

在一个始终在线的代理中，代理处于休眠状态，直到被唤醒，其循环运行在服务器上。

一个始终在线的代理会一直处于休眠状态，直到收到消息或触发信号将其唤醒，然后通过启发式算法来决定是否值得向用户推送更新信息。

第三部分，让我们更深入地了解一下这个始终在线的代理的运作机制，或者说幕后运作。代理始终在线意味着什么？你可以从左到右阅读这张图表，它从服务器处于休眠状态开始，代理可以使用的计算机也处于休眠状态。无论是你发送消息还是某种触发器决定启动代理（我们稍后会讨论），这基本上就是加载和启动代理所有状态的过程，而它运行在服务器上。如果它决定需要使用计算机，例如它想使用计算机的浏览器或在计算机上进行一些操作，它也可以启动计算机。然后，一旦使用完毕，它也可以将这些计算机置于休眠状态。

为什么这很重要？因为这会很昂贵。世界上的计算机数量有限，无法让所有这些计算机一直为所有人运行，同时也会占用大量的服务器资源。因此，可以将这些状态（包括对话记录、记忆、例程等等）持久化到数据库中。

那么，如何唤醒机器人呢？当然，你可以直接发出指令，但越来越有趣的是，让其他程序在后台运行，然后根据你的需要决定何时联系机器人。比如，有人在 Slack 上联系你，而你的机器人可以驻留在 Slack 中。又比如，你安排机器人参加电话会议或会议。在 SpaceX，这种情况越来越普遍：员工本来可以亲自到场，但他们希望机器人代为参加会议并做笔记，然后再返回给你，这其实挺有意思的。也许你有一些例行工作或后台工作需要花费数小时甚至数天才能完成，然后你想让机器人回来给你一份总结。这些情况都可能发生。

然后我们基本上有一个路由器，它会对这些信息进行去重，并确定如何将其融入对话中。之后，它会运用一种启发式方法：我是否需要就此事打扰用户？因为并非所有事情都真正重要。就像你的同事在做某件事，如果他每次取得进展都来烦你，那会非常烦人。你真正需要的可能只是每天一次的状态更新，具体取决于任务。

代理的 Firecracker 虚拟机很容易创建快照，并且将代理循环保留在服务器上，这样就可以在不停机的情况下重建该机器。

"我讲了代理程序如何利用这台计算机，这台 Linux 服务器。它运行着一个 Firecracker 虚拟机。这样做的好处在于，可以很方便地对机器状态进行快照，然后将其关闭并重新启动。这台机器上当然可以存放文件，但你也可以连接到它，实时查看机器状态，并根据需要进行控制。这台机器当然可以运行命令，如果你要运行一些 shell 命令，那就非常棒了。也许你不想在你的个人笔记本电脑上运行这些命令，而是想在一个安全且受保护的环境中运行。另一个有趣的地方是，它当然可以安装 Chrome 浏览器，但你也可以为它设置虚拟桌面。所以，如果你运行了 5 个、10 个机器人，每个机器人都需要自己的浏览器，你可以让它们拥有不同的屏幕。所有这些都是这些机器人之间共享的基础架构，它们可以调用这些计算机。"

我们之前讨论过唤醒代理的触发器，也就是左侧的那些，比如不同的应用程序、Slack 消息或电子邮件。中间部分是服务器，右侧是我们刚才提到的计算机，你可以在上面运行文件、命令等等。现在我们再详细说说这个服务器。命令会进入队列，然后服务器会调用紫色框，也就是代理循环，这是我们上周讲过的——这是一个循环调用工具的模型。关闭代理时，你可以将状态保存到数据库或存储中。

我觉得很有意思的是，这个看似不起眼的小组件，在其上构建了所有这些抽象层，最终形成了一个功能强大的产品，因为它配备了所有这些有用的工具，并且连接到了所有这些不同的服务。

代理程序运行在服务器上是计算机领域一种相当常见的做法，它类似于瘦客户端和胖服务器的组合，而计算机领域多年来一直在这两种模式之间摇摆不定。这样做有几个很好的理由：假设你需

要更新代理程序代码，如果为了发布更新而不得不推送应用商店更新，那可就太麻烦了，还不如直接快速地推送一系列服务器更新。此外，有时你需要执行一些繁重的计算任务，比如转录、图像生成等等，这些任务在服务器上运行的速度可能比在设备上更快，具体取决于设备类型。而且，我们已经掌握了非常成熟的服务器扩展方法，尤其是在流量和增长不稳定的情况下。但最关键的是，如果你把代理程序循环放在虚拟机上，那么你就无法独立于代理服务器关闭虚拟机。

所以，你需要把所有东西都保存在服务器上，这样你就可以随时重建虚拟机。例如，假设你需要对虚拟机进行一些关键的零日 Linux 更新，那么能够重建虚拟机而不至于破坏整个代理循环、导致应用程序停机，就非常重要了。

持久工作流会在任务崩溃后从停止的步骤恢复执行，并且用户自己的消息优先于正在发生的所有其他事情。

持久工作流而不是作业队列，然后是新消息的排序规则

因此，这里需要考虑的底层基础设施是工作流和作业队列。主要区别在于，假设使用作业队列时，你正在按步骤执行任务。你让代理执行一些复杂的任务，它会依次执行第一步、第二步和第三步。但是，由于某种原因，基础设施出现了问题。如果使用普通的作业队列，当它崩溃并需要重启时，你就必须重新执行第一步、第二步和第三步，这显然会非常麻烦。

所以，你真正需要的是一个持久的工作流程，这意味着你可以安全地重试。当失败后重试时，它会直接从第三步开始。在分布式系统和基础设施领域，这在某种程度上已经是一个比较成熟的问题了。我们使用了一个名为 Temporal 的开源工具，如果你感兴趣，可以了解一下。这样我们就不用从头开始重建很多非常复杂的功能了。当然，你仍然需要运行相关的基础设施，但有些组件已经集成好了。

假设收到一条新消息。根据消息的发送对象（例如用户、机器人、群聊、Slack 讨论串或后台程序），优先级会有所不同。比如，如果我说"哦，实际上，需要更改这个活动的日期"，我们希望代理的循环停止，我的消息优先级高于所有其他正在处理的消息，这很合理。对于其他消息，优先级会排在队列末尾。如果用户很快收到三四条消息，它可以将它们集中处理，这样就只需要处理一次。你可以根据自己的需要设计任何启发式算法，然后系统会依次处理每个消息。

该连接方式在客户端和服务器之间保留了一个工具，并将繁重的工作交给辅助工具完成。

只有一个工具（发送给用户），连接 Grok Bot 客户端和服务器，每个客户端决定如何显示返回的内容。

"好的，我们来谈谈这个框架本身。它会在我们上周讨论的基础框架的基础上添加一些新工具和新概念。Grok Bot 框架与市面上其他类似工具最大的区别在于，我们采用了一种瘦客户端固定服务器架构，客户端和服务器之间只需要一个工具进行通信。这个工具的作用就是向用户发送数据。这种架构的优点在于它非常简洁，客户端只是以某种方式向服务器发送消息，而将许多更复杂的工具放在服务器端运行。这些工具种类繁多，例如运行 shell 命令、读取文件、调用云代理、录制屏幕或进行演示等等。我认为这种架构运行良好，因为现在无论你在哪里启动一个新客户端，无论是移动应用、服务器、桌面应用、电子邮件还是 Slack，都只需要这一个工具即可。它与服务器进行双向通信。

不过，在客户端，即使只有一个工具，当它接收到该工具时，也能决定如何以最符合用户需求的方式，用丰富且交互性强的界面呈现。例如，如果它从服务器收到用户尝试登录的信息，就可以显示独特的登录表单，并连接到像 1Password 这样的工具。它还可以调用 Stripe Link 或 X Money 等服务，让用户支付服务费用。如果你尝试执行一些无法撤销的操作，例如发送电子邮件或 Slack 消息，系统可能会要求你编辑并批准草稿后再发送。有时，如果你不需要发送消息，也可以直接留下表情符号作为回应。"

预先加载每个工具的详细信息会使上下文窗口变得拥挤，因此代理会先获取工具名称，并在打扰用户之前尝试最便宜的选项。

打造出一个感觉拥有无限上下文的产品

"回到我刚才提到的动态工具发现，对于任何模型来说，都有一个上下文窗口，它就像是模型的工作内存，而上下文窗口的容量是有限制的。如果你把所有可能的工具都塞进这个上下文窗口，空间就会非常有限。工具的工作方式是这样的：你拥有工具名称、模式以及各种不同的细节信息。如果你开始安装 Google Drive 连接器、Gmail 连接器、YouTube 连接器以及所有这些使用 MCP 服务器（它们都包含工具）的服务，你会发现顶部区域的大部分空间已经被填满了。问题在于，这会占用模型足够的空间来真正处理对话。"

所以，现在很多流行的模型框架都使用一种上下文工程或框架技巧，叫做动态上下文，或者有些人称之为工具搜索工具，这个名字挺有意思的。简单来说，就是把工具的名称添加到发送给模型的每条消息中，然后模型就可以决定，比如，如果我想用这个工具，可以从文件中读取信息。所有这些信息都存储在文件系统中。这样做可以节省上下文的使用，从而降低成本，减少资源消耗。

总的来说，这里的原则是先尝试最便宜或最可靠的方案，然后逐步升级。最简单的方案就是利用代理或机器人基于对话历史已有的信息。如果信息不在当前上下文中，可能需要使用插件、MCP 服务器或某种 API，例如，它可以从 Plaid 获取您的银行信息。如果这种方法行不通，它就需要进行网络搜索，例如查找昨晚小熊队的比赛比分。如果仍然找不到，它可能需要访问用户的电脑，打开浏览器进行搜索。再进一步，它可能需要访问整个桌面，例如运行脚本、命令或在 Linux 电脑上构建程序。只有当所有这些方法都失败后，才需要联系用户，请求他们提供输入或批准某些内容。

这确实有助于营造与同事合作的氛围。他们不会因为各种各样的事情而打扰你，理想情况下，他们会先尝试上述这些方法中的很多种。

主代理通过将繁重的工作交给子代理来保持自身对话的简洁，这使得产品能够给人一种拥有无限上下文的感觉。

"在那台电脑上，也就是代理能够使用的那台 Linux 机器上，它可以通过三种不同的方式从中检索信息。当然，它可以直接截屏，但还有更高效的方法。它还可以查看页面的 HTML 代码，或者查看可访问性树，这是一种减少令牌使用量的有效方法，基本上可以帮助你找到你想让浏览器点击的按钮、链接或其他元素。当然，如果这些方法都不奏效，你还可以直接点击像素。例如，有些网站会弹出模态框或对话框，你需要点击它们，因此你的模型，也就是你的代理，也需要具备这种功能。或者，最糟糕的情况是，你回到用户那里，发现需要双因素身份验证码或某种验证码之类的东西。"

把所有组件整合起来，你有一个主代理，它就像你的协调器。你要尽可能地保持主代理的对话简洁，因为它现在可以调用各种不同的辅助代理，而这些辅助代理只是子代理。这样做的好处在于，如果你把任务交给辅助代理去运行一堆 shell 命令或者进行很多耗时的工具调用，所有这些最终都会得到问题或任务的答案，但这些都不会增加主代理对话的复杂性，它只会返回结果。这样就能打造出一个感觉拥有无限上下文的产品。

当然，无限上下文并不存在。每个模型都有一个上下文限制，但是有一些技巧和变通方法，尤其是在训练模型以提高其处理能力时，不会感觉遇到了上下文窗口的限制，也不会感觉模型的质量随着训练的深入而下降。

所以我们有不同的助手，每个助手都具备不同的技能，每个助手都有自己独立的持久工作流。回到队列和工作流的区别，这意味着如果某个助手出现故障（例如视频助手崩溃、内存占用过多等等），它可以重启并继续运行，而不会丢失任何进度。而主代理则有点像同事，负责管理待办事

项列表。你可以对它执行一些操作，例如查看进度、发送消息、停止助手等等。因此，这套系统很大程度上是模拟了如何扩展一个工程师团队来解决一个非常复杂问题的理想方式。

上下文工程通过缓存提示信息、保持主对话简洁以及分层内存管理，实现提示信息的缓存和主对话的精简。

理想情况下，代理的大部分令牌都会被缓存，因此工具定义和系统提示保持不变，而冗长的输出和繁重的工作则会写入文件和辅助程序。

"上下文工程也称为框架工程。它们随着时间的推移而不断发展。基本上，我们可以采取哪些技巧来最大限度地减少这些框架中使用的上下文量，从而提高效率、降低成本并带来其他好处？我之前提到过提示缓存。这是一个很好的例子，说明了它为何如此重要。想想看，当你使用编码代理或任何代理时，理想情况下，大多数使用的令牌实际上都是缓存令牌。因为你发送了一段对话，然后你进行了更改，添加了一条消息，然后基本上又重新发送了整个对话。所以，如果你查看模型的定价，当然，如果令牌被缓存，价格会有很大的折扣。而且你也希望尽可能长时间地将它们保留在缓存窗口中。"

因此，一种方法是尽量保持工具定义、系统提示等所有信息在不同调用之间尽可能保持不变，这样就不会破坏缓存，从而避免每次都为未缓存的令牌付费。如果顶部的固定部分保持不变，那么当调用到底部，只剩下最新收到的消息时，就只需要为这部分差异支付未缓存的令牌费用。这其中也蕴含着丰富的技巧和学问。

基本上，主代理（在最理想的情况下，也就是你与之进行非常非常长时间对话的那个代理）会不断地压缩或概括上下文，有时你甚至不会注意到，我们稍后会举例说明。所以你真的需要保持上下文简洁。一些方法可以做到这一点：当输出内容很长时，与其将其塞进上下文，不如将其放入一个文件中。代理非常擅长读取文件，所以这种方法效果很好，例如读取技能或记忆。如果你给模型发送截图或其他非常耗时的任务，只需将其委托给辅助代理即可。这样，任务就会交给子代理，而不会影响主代理的上下文。而且，在理想情况下，你还应该不断地修剪、整理，从对话、从这个上下文窗口中移除不必要的内容，这样做既是为了降低成本，也是为了便于记忆，并确保模型运行良好。

缓存未命中就像一个 bug，趁缓存还热的时候压缩一段很长的对话，可以得到一个更便宜的摘要，以便在此基础上继续构建。

"所以，如果出现缓存未命中，实际上就相当于系统出现了一个 bug。回溯到今年 1 月，当时 OpenClaw 正迅速流行起来，人们开始在模型提供商的订阅计划中使用它。这是一种模型提供商之前从未见过的全新使用模式。而且，当时的框架还没有针对这种工作流程进行优化。因此，缓存未命中的情况非常普遍。而当代理程序持续运行时，这显然会造成问题。所以，我们做了大量的修复工作，以确保系统能够尽可能地保持提示信息的缓存。"

这里有个小技巧，就是尽量保持提示信息中的所有内容有序。这样，即使你部署了新的更改，想要测试系统提示信息的新变化或添加新工具，你也可以对每个部分进行哈希处理，始终保持固定的顺序。假设你正在进行 A/B 测试，想要添加提示信息的新内容，你肯定不希望这打乱其他用户的提示信息顺序，否则就会破坏缓存。另一个有趣的技巧是，当你打开聊天窗口并开始输入时，服务器可以提前准备好提示信息，相当于预热缓存，就像你在网站上将鼠标悬停在链接上时，服务器会预取下一页的内容一样。因此，有很多小的工程优化可以提升这些模型在令牌使用方面的效率，并增强它们在长时间使用时的稳定性。

关键在于，当你长时间工作时，你必须非常擅长压缩或摘要。假设模型有一个包含 20 万个令牌的上下文窗口，你会一直处理下去，最终，它会遇到需要进行摘要的情况。而所有压缩都是有损的，所以你必须找到最佳的压缩算法，既能有效压缩，又能避免丢失重要的细节。这真的非常非常难。

举个例子，假设你正在和你的机器人进行交互，你积累了超过 10 万个令牌，然后你去做其他事情。这时，模型提供商会有一个缓存窗口期，在此期间缓存仍然有效。这个窗口期可能是 10 分钟，也可能是 60 分钟。我们有一个技巧，我认为非常有用：在缓存有效期内，你应该立即进行压缩，因为对话时间很长，压缩成本会低得多。所以，现在就进行压缩，当用户 30 分钟后回来时，他们将从这个摘要开始，继续添加消息的成本会低得多。因此，我们需要做很多工作来提高摘要算法的质量。

代理的记忆是分层的，从每次都包含的核心事实到短暂的草稿笔记，其他一切都通过文件查找。

"谈到信息压缩以及选择哪些信息存储在智能体的内存中，我通常会从几个不同的层面来考虑。首先是关于用户的重要信息，这些信息必须每次都包含在内。就像我之前提到的，用户总是希望看到小写文本，这可能需要出现在每条消息中，否则，如果某条消息突然出现大写字母，那就太奇怪了，就像 Sam Altman 那样。此外，你可能还需要包含一些提示信息或一些想要存储的信息。有很多不同的方法可以设计算法来确定哪些信息最重要，需要保留。理想情况下，应该对这些信息进行某种形式的垃圾回收。"

仔细想想，我们的大脑就是这样运作的。比如，某个事实现在很重要，需要记住，它与今天的对话相关；你现在可能不会想到足球比赛。你的大脑很擅长把这些信息放在当前情境中，以便日后使用。我们尝试在这里模拟一下，这个系统当然包含这些核心事实。它还记录了你过去的对话历史，并附有日期。此外，它还有一些类似草稿纸的笔记，记录的是你现在正在处理的一些小事，它们被设计成短暂的，会消失，但可能在接下来的 30 分钟或 1 小时内仍然有用。其他所有信息都可以通过文件找到。这就是文件的神奇之处，也是模型擅长执行命令的原因——它们可以随时查找信息，而且它们在这方面做得非常出色。

从长远来看，模型和产品会同步改进，而尚未解决的问题则始于运行一个月的代理。

理想情况下，代理的失败会在每日内循环中成为评估，而在外循环中，新模型会从中学习并提升评估结果。

未来6到12个月代理将能连续工作数月

这种飞轮机制的运作方式是这样的：假设你的智能体出错了。理想情况下，这会被转化为某种评估。比如，模型意外地调用了某个工具，而它本不应该这样做。然后在后端，你可以修复框架、修复错误、修复提示，或者可能是推理问题。你可以在评估中检查并确保问题确实已修复，然后进行相应的更改。负责这些产品的团队几乎每天都在做这件事：他们发现错误，修复错误，并通过评估来衡量结果。这是一个不断改进的内部循环。

但还有一个外循环，大概每隔几个月或者一个月，你都会训练新的模型。理想情况下，新模型会从所有这些错误和失败中学习，并在评估中不断进步，从而在你关心的方面做得更好。你可以将新版本部署到产品中，确保它们在各个方面都有所改进。有趣的是，当新模型发布时，它们会带来全新的功能飞跃，让你能够以全新的方式使用产品。

例如，目前还没有太多关于在群聊中使用机器人的训练数据。这算是一个正在探索的新兴领域，而确保其正确运行至关重要。显然，如果你看过像 Moltbook 或 Hugging Face 这样的案例，就会明白其中存在一些失败的案例。因此，训练模型如何在多智能体系统中良好运行至关重要，例如，在不同的机器人相互交流的情况下。理想情况下，作为一家致力于人工智能的公司的产品团队，在构建这类框架时，应该始终运行这些内部循环和外部循环。

如果把今天的模型视为它们有史以来最糟糕的情况，那就意味着要尽可能少地构建，他认为代理工作数周或数月的情况很快就会出现。

"我们在 Cursor 和现在的 SpaceX 都奉行的一个原则是'删除产品'。这意味着你必须接受并假设今天的模型已经是它们未来最糟糕的状态，也就是说，六个月后你可能需要彻底重建用户界面。

我今天谈到的大部分内容，与六个月前甚至一年前相比，都是相当大的更新，因为我们学习到了很多构建模型框架和进行上下文工程的实用新方法。所以，**一个好的策略是，努力为下一代模型构建框架——如何在尽可能少地构建的同时，打造一个极其简单的界面来与这些模型交互，并让它们为你思考和行动？"**

我认为未来六到十二个月内会发生一些事情，我们拭目以待。就目前而言，假设你的代理或机器人可能每天工作几个小时，也可能一天，具体取决于任务的难度。我认为，你的代理很可能很快就能连续工作数周甚至数月。这确实引发了一些关于如何训练和评估这些模型的有趣问题。我认为，模型在回忆对话中的历史信息方面会越来越出色。目前，针对所使用的算法类型或底层数据表示方式，研究领域有很多不同的方法，也许是图，也许是文件，也许是其他形式。

我认为理想情况下，如果你有这样一个数字化同事，你应该像培训真人一样，为它提供所有必要的工具。当然，我认为各个组织仍在逐步接受这个理念，所以还有很多工作要做。我认为界面可能会变得更加便捷，有些人可能更喜欢用语音交互的方式与机器人或智能代理交流，他们或许会拥有乔纳森·艾维设计的炫酷设备，可以整天和它对话。我认为模型和产品会越来越擅长在工作中学习技能，并将这些技能编码成文件、指令和技能文档，从而将这些原始智能转化为实际应用，完成你交给它的任何任务。**我认为，人们和公司会越来越擅长记录所有这些定制流程，以便智能代理能够理解和使用它们。**

新产品要么集成到现有系统中，要么就沦为供其他代理使用的数据

"所以，我认为这里有一些有趣的问题，或许能启发大家进行研究或探索。首先，我们回到之前提到的运行一个月的智能体。当需要评估一个运行一个月的智能体，而模型却每月发布一次时，该怎么办？这样就无法全面评估模型是否完全符合预期，所以要么需要找到更好的评估方法，要么就得降低模型的发布频率。我认为这将是未来六个月内整个行业必须解决的问题。"

安全方面，我们讨论过不同的算法，我认为还有很多工作要做，比如如何构建安全的框架，如何确保这些代理的基础设施非常安全，从而防止它们被恶意攻击者控制。我们可能没有过多讨论代理何时保持沉默或发出提醒，这方面的启发式方法我们还没有做到完美。我认为，如何在主动提醒和主动告知之间找到平衡点，比如"嘿，我看到你昨天缺课了，我录了课并做了笔记"，这或许对你有帮助，或许会有点烦人，我们需要找到合适的平衡点。

我也认为，可以相当肯定地假设，到明年这个时候，世界上流通的令牌数量将会显著增加，这意味着优化推理、优化模型、优化工具，以及努力降低令牌使用量以提高效率，都将非常重要。

所以现在需要思考一些新的构建模块。我记得在 X 大会上看到 OpenRouter 的 Alex 对此有非常精辟的见解。有人说，现在所有应用看起来都一样：侧边栏、代理、聊天框，大家都在做同样的事情。Alex 说，是啊，那就像 2005 年一样，大家都在做同样的事情，他们有服务器、有数据库、有用于更新和删除数据的 CRUD 界面。**而现实是，这已经成为一种新常态，每个新产品都必须具备某种代理功能，要么集成到现有系统中，要么就沦为供其他代理使用的无头数据。**

今天我们讨论了很多这类产品，比如持久工作流、沙箱、云端计算机、内存、技能、与其他服务的连接、语音、支付、身份验证、评估、可观测性等等。每一个领域都相当于数十亿甚至数万亿美元的风险投资机会，每个领域都有二十家初创公司。还有很多不同的东西值得探索，因为它们可以说是下一代公司、下一代产品的基础组件。它们都必须整合起来，成为各种工具的一部分。所以，这方面还有很多工作要做。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

138 收藏 分享到:

写评论

请发表您的评论

表情 图片 发布评论

1 条评论



deus

4小时前 中国重庆

本地能做的事，却要放到服务器上去，疯了吧

0 回复